DESIGN AND VERIFICATION OF AN AXI4-LITE BASED CRYPTOGRAPHIC ACCELERATOR

Project Report

Submitted in partial fulfilment of the

requirements of the Summer Internship

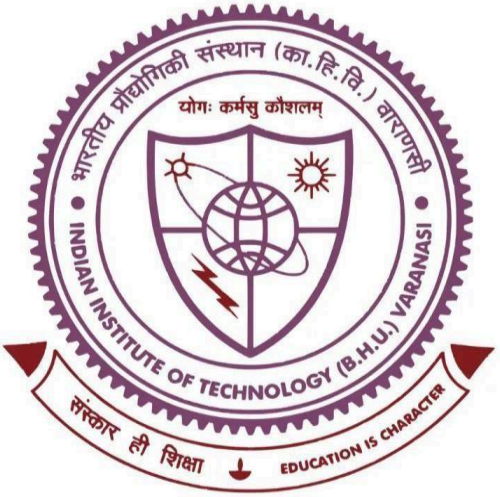
Programme

by

Alok Jiten Rao
Biswajeet Sahoo
Shubhendu Sa
Nanak Ghosh
Sanskriti Mishra

under the guidance of

Dr. Shivam Verma (Assoc. Professor)



DEPARTMENT OF ELECTRONICS ENGINEERING,

INDIAN INSTITUTE OF TECHNOLOGY (BHU), VARANASI

UTTAR PRADESH – 221005, INDIA

JULY – 2026

TABLE OF CONTENTS

S No.	Name of the chapter	Page no.
1.	Abstract	3
2.	**Chapter 1:** Introduction	4-5
3.	**Chapter 2:** Design Of AXI4-LITE Based Cryptographic Accelerator	6-10
4.	**Chapter 3:** Working and Verification Of AXI4-LITE Based Cryptographic Accelerator	11-16
5.	Conclusion	16
6.	References	16

ABSTRACT

With the increasing demand for secure data communication in modern embedded systems and System-on-Chip (SoC) architectures, hardware-based cryptographic accelerators have become essential for achieving high-speed and efficient encryption. This project presents the design and verification of an **AXI4-Lite Based Cryptographic Accelerator** by integrating an Advanced Encryption Standard (AES-128) encryption core with an AXI4-Lite slave interface using Verilog HDL.

The proposed design enables a processor or host system to communicate with the AES encryption engine through memory-mapped registers. Plaintext and encryption keys are loaded into dedicated registers via AXI4-Lite write transactions, while a control register initiates the encryption process. A Finite State Machine (FSM) manages the overall operation by controlling the AES core, monitoring the encryption status, and storing the generated ciphertext into output registers. A status register indicates the completion of the encryption process.

The complete design was implemented and functionally verified using the Xilinx Vivado Design Suite. Behavioral simulation confirmed the correct operation of the AXI4-Lite write interface, register mapping, FSM transitions, AES encryption process, and ciphertext generation. Waveform analysis demonstrated successful communication between the AXI interface and the AES core, validating the correctness of the implemented architecture.

The developed cryptographic accelerator provides a modular, scalable, and efficient hardware solution for secure data processing. The project demonstrates the practical integration of standard bus protocols with cryptographic hardware and serves as a foundation for future FPGA-based secure embedded systems and VLSI hardware security applications.

CHAPTER

1

INTRODUCTION

1.1 Objective

In modern digital systems, secure communication has become a critical requirement due to the rapid growth of embedded systems, Internet of Things (IoT) devices, cloud computing, and high-speed data networks. Cryptographic hardware accelerators provide fast and secure data encryption while reducing the computational burden on processors. Unlike software-based encryption, hardware implementations offer lower latency, higher throughput, improved security, and reduced power consumption.

This project focuses on the **Design and Verification of an AXI4-Lite Based Cryptographic Accelerator** using Verilog HDL. The accelerator integrates an **AES-128 encryption core** with an **AXI4-Lite slave interface**, enabling a processor or host system to communicate with the encryption engine through memory-mapped registers.

The operation of the cryptographic accelerator consists of the following major stages:

- **Register Configuration:** Plaintext and encryption key are written into dedicated registers through AXI4-Lite write transactions.
- **Control Operation:** The processor initiates encryption by writing to the control register.
- **Encryption:** The AES-128 core performs hardware-based encryption under the supervision of a Finite State Machine (FSM).
- **Status Monitoring:** The FSM updates the status register after encryption is completed.
- **Result Retrieval:** The encrypted ciphertext is stored in output registers and can be accessed by the processor.

The complete hardware design has been implemented in **Verilog HDL** and verified through behavioral simulation using **Xilinx Vivado Design Suite**.

1.2 Motivation

The increasing demand for secure embedded systems and System-on-Chip (SoC) architectures has created a need for dedicated cryptographic hardware. Implementing encryption directly in hardware provides several important advantages over software-based solutions.

High-Speed Encryption

Software encryption consumes processor cycles and increases execution time. A dedicated AES hardware accelerator performs encryption independently, significantly reducing processing latency and improving overall system performance.

Standard AXI4-Lite Communication

Modern FPGA and SoC designs commonly use the AXI protocol for communication between processors and peripherals. Integrating the AES core with an AXI4-Lite interface allows seamless communication using memory-mapped registers, making the design compatible with ARM-based processors and FPGA systems.

Efficient Hardware Resource Utilization

The project employs a finite state machine (FSM) to control encryption operations efficiently. The FSM manages data loading, encryption start, status monitoring, and ciphertext generation while minimizing hardware complexity.

Hardware Security

Executing cryptographic algorithms in dedicated hardware reduces processor workload and minimizes exposure of sensitive operations to software attacks. Hardware accelerators also provide deterministic execution timing and improved reliability for secure applications.

Modular and Scalable Design

The architecture is modular, allowing the AES encryption core to be integrated with different communication protocols or extended to support additional cryptographic algorithms in future work.

FPGA and VLSI Compatibility

The entire accelerator is described in synthesizable Verilog HDL and verified using Xilinx Vivado. The design methodology is compatible with FPGA implementation and serves as a foundation for future ASIC realization in VLSI design flows

CHAPTER

2

DESIGN OF AXI4-LITE BASED CRYPTOGRAPHIC ACCELERATOR

2.1 Basic AXI4-Lite Cryptographic Accelerator

The proposed cryptographic accelerator integrates an **AXI4-Lite slave interface** with an **AES-128 encryption core** to provide a secure and efficient hardware encryption solution. The AXI4-Lite interface serves as the communication bridge between the processor (master) and the AES hardware accelerator (slave), enabling data transfer through memory-mapped registers. Using this interface, the processor can write the plaintext and encryption key into the accelerator, initiate the encryption process, monitor the execution status, and retrieve the generated ciphertext after encryption is completed.

The complete architecture is composed of the following major functional blocks:

- AXI4-Lite Slave Interface
- Control Register
- Status Register
- 128-bit Plaintext Register
- 128-bit Key Register
- AES-128 Encryption Core
- 128-bit Ciphertext Register
- Finite State Machine (FSM)

The processor communicates with these hardware blocks through dedicated memory-mapped registers. Each register is assigned a unique address, allowing data to be transferred using standard AXI4-Lite write and read transactions.

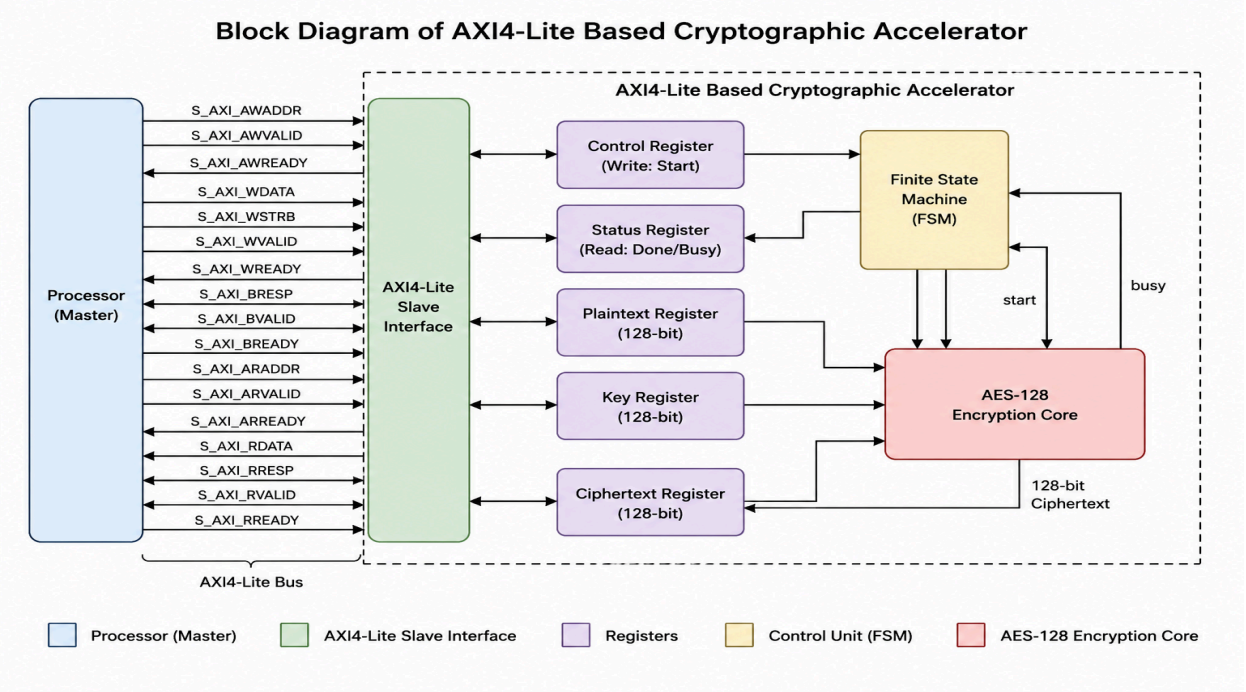


Fig. 2.1: Block Diagram of AXI4-Lite Based Cryptographic Accelerator

The proposed cryptographic accelerator consists of an AXI4-Lite slave interface integrated with an AES-128 encryption core. The AXI4-Lite slave interface receives commands and data from the processor through memory-mapped transactions. The plaintext and encryption key are first stored in their respective registers. After the control register receives the start command, the Finite State Machine (FSM) activates the AES-128 encryption core. During encryption, the FSM continuously monitors the aes_busy signal generated by the AES engine. Once the encryption process is completed, the generated ciphertext is stored in the ciphertext register, and the status register is updated to indicate successful completion. Finally, the processor retrieves the encrypted output through the AXI4-Lite interface.

2.2 AXI4-Lite Interface

AXI4-Lite (Advanced eXtensible Interface Lite) is a lightweight version of the AMBA AXI protocol developed by ARM for low-bandwidth peripheral communication. It provides a simple and efficient memory-mapped interface for transferring control and status information between a processor and peripheral devices. Due to its low hardware complexity and ease of implementation, AXI4-Lite is widely used in FPGA-based embedded systems and System-on-Chip (SoC) designs.

The AXI4-Lite protocol separates write and read operations into independent communication channels, allowing efficient and reliable data transfer.

Write Address Channel (AW)

The write address channel transfers the destination register address from the processor to the

peripheral. The processor places the register address on the **AWADDR** bus and asserts the **AWVALID** signal to indicate that the address is valid. The slave acknowledges the address by asserting **AWREADY**, completing the write address handshake.

Write Data Channel (W)

After the address transfer, the processor places the data on the **WDATA** bus and asserts **WVALID**. When the slave is ready to receive the data, it asserts **WREADY**. The write data transfer is completed only when both **WVALID** and **WREADY** are asserted simultaneously.

Write Response Channel (B)

Once the write transaction is successfully completed, the slave generates a write response by asserting **BVALID**. The processor accepts this response by asserting **BREADY**, indicating that the write operation has been completed successfully.

Read Address Channel (AR)

For a read operation, the processor places the target register address on the **ARADDR** bus and asserts **ARVALID**. The slave acknowledges the read request by asserting **ARREADY**, thereby accepting the requested address.

Read Data Channel (R)

After processing the read request, the slave places the requested register contents on the **RDATA** bus and asserts **RVALID**. The processor receives the data by asserting **RREADY**, completing the read transaction.

2.3 Register Organization

The proposed cryptographic accelerator employs memory-mapped registers for communication between the processor and the AES encryption engine. Each register performs a dedicated function during the encryption process.

Control Register

The control register is responsible for initiating the encryption operation. Writing a logic '1' to the start bit generates a start pulse that activates the AES encryption core through the FSM.

Status Register

The status register indicates the current operating status of the accelerator. Once the AES encryption process is completed, the FSM updates this register to notify the processor that the ciphertext is available.

Plaintext Register

The plaintext register stores the 128-bit input message to be encrypted. Since the AXI4-Lite interface supports a 32-bit data bus, the plaintext is loaded into four consecutive 32-bit

registers.

Key Register

The key register stores the 128-bit AES encryption key. Similar to the plaintext register, the encryption key is transferred into the accelerator through four successive 32-bit write transactions.

Ciphertext Register

After successful encryption, the AES core generates a 128-bit ciphertext which is stored in the ciphertext register. The processor can retrieve the encrypted output through AXI4-Lite read transactions.

2.4 Finite State Machine (FSM)

The overall operation of the cryptographic accelerator is controlled by a Finite State Machine (FSM) consisting of three operational states.

IDLE State

During the IDLE state, the accelerator waits for the processor to load the plaintext, encryption key, and start command. No encryption activity takes place in this state.

RUN State

After receiving the start command from the control register, the FSM generates a single-clock start pulse for the AES core. The AES engine begins encryption while the FSM continuously monitors the **busy** signal to determine the progress of the encryption operation.

DONE State

Once the AES core completes encryption, the FSM stores the generated ciphertext into the ciphertext register and updates the status register to indicate successful completion of the encryption process. The accelerator then remains in the DONE state until the next encryption request is issued.

2.5 Working Principle

The complete operation of the proposed cryptographic accelerator is summarized as follows:

1. The processor writes the 128-bit plaintext into the plaintext registers using AXI4-Lite write transactions.
2. The 128-bit AES encryption key is loaded into the key registers.
3. The processor initiates encryption by writing to the control register.
4. The FSM generates a start pulse and activates the AES-128 encryption core.
5. The AES engine performs hardware-based encryption while asserting the busy signal.
6. After encryption is completed, the generated ciphertext is stored in the ciphertext register.

7. The FSM updates the status register to indicate successful completion.
8. Finally, the processor retrieves the encrypted ciphertext through AXI4-Lite read transactions.

The proposed architecture provides a secure, modular, and efficient hardware encryption solution. The use of the AXI4-Lite protocol enables seamless integration with modern FPGA and System-on-Chip (SoC) platforms, while the modular Verilog implementation allows the design to be extended for future cryptographic and embedded system applications.

.

CHAPTER

3

WORKING AND VERIFICATION OF AXI4-LITE BASED CRYPTOGRAPHIC ACCELERATOR

3.1 Working of the Proposed Design

The proposed AXI4-Lite Based Cryptographic Accelerator performs AES-128 encryption under the control of an AXI4-Lite slave interface and a Finite State Machine (FSM). The processor communicates with the accelerator using memory-mapped registers to transfer the plaintext, encryption key, and control information.

Initially, the processor loads the 128-bit plaintext and the 128-bit encryption key into their respective registers using AXI4-Lite write transactions. Since the AXI4-Lite data width is 32 bits, both the plaintext and the key are transferred in four consecutive write operations. After loading all the required data, the processor writes a logic '1' to the control register, which generates a start command for the encryption process.

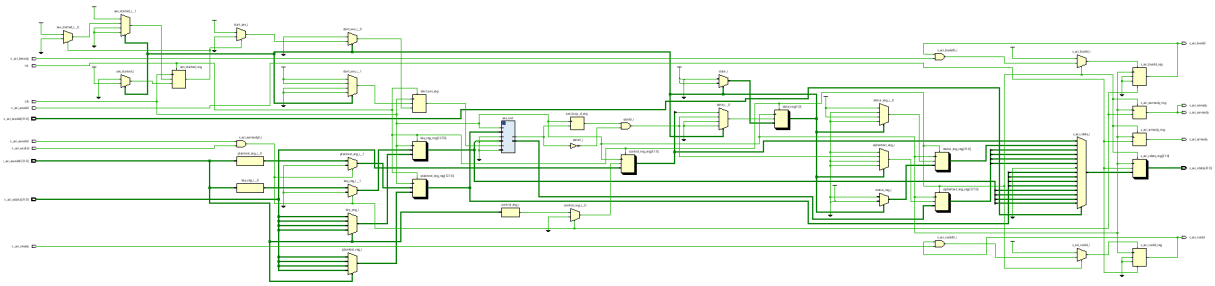


Fig. 3.1: RTL Schematic of the Proposed AXI4-Lite Based Cryptographic Accelerator

The RTL schematic illustrates the overall hardware organization of the proposed AXI4-Lite Based Cryptographic Accelerator. It includes the AXI4-Lite slave interface, control and status registers, plaintext and key registers, Finite State Machine (FSM), AES-128 encryption core, and ciphertext register. The schematic confirms the correct interconnection of all functional modules and verifies the hardware implementation of the proposed design.

The Finite State Machine (FSM) detects the start command and transitions from the **IDLE** state to the **RUN** state. During this state, a single-clock start pulse is generated for the AES encryption core. The AES engine begins encrypting the input plaintext using the supplied

128-bit key while asserting the **busy** signal to indicate that encryption is in progress.

Once the AES core completes encryption, the **busy** signal is de-asserted. The FSM detects this transition, stores the generated ciphertext into the ciphertext register, updates the status register to indicate successful completion, and finally enters the **DONE** state. The encrypted output can then be accessed through AXI4-Lite read transactions.

3.2 AXI4-Lite Write Transaction Verification

Behavioral simulation was performed using Xilinx Vivado to verify the correct operation of the AXI4-Lite interface. During the write operation, the processor first transfers the destination register address through the **AWADDR** bus and asserts **AWVALID**. The accelerator acknowledges the address by asserting **AWREADY**. The corresponding data is then transferred through the **WDATA** bus using the **WVALID/WREADY** handshake. After successful storage of the data, the accelerator generates a write response through the **BVALID/BREADY** handshake.

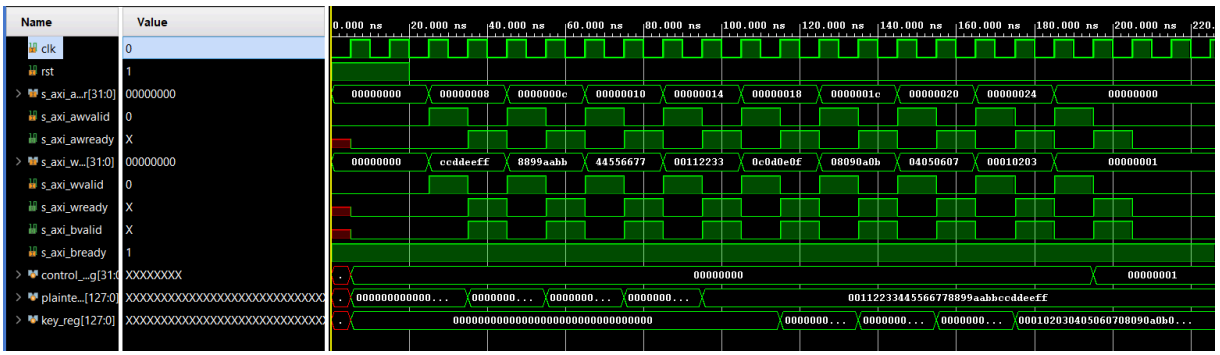


Fig. 3.2: AXI4-Lite Write Transaction Waveform

The AXI4-Lite write transaction illustrates the loading of the plaintext, encryption key, and control information into the accelerator. During each transaction, the write address (`AWADDR`) and write data (`WDATA`) are transferred through the AXI4-Lite interface using the `AWVALID/AWREADY` and `WVALID/WREADY` handshake signals. After successful data transfer, the accelerator generates a write response through the `BVALID/BREADY` handshake. The waveform confirms that all write transactions are completed successfully before the next transaction begins, ensuring reliable communication between the processor and the cryptographic accelerator.

3.3 Finite State Machine Verification

The operation of the accelerator is controlled by a three-state Finite State Machine consisting of **IDLE**, **RUN**, and **DONE** states. Initially, the accelerator remains in the IDLE state waiting for the start command. After receiving the start signal, the FSM enters the RUN state and activates the AES encryption core. When encryption is completed, the FSM stores the ciphertext and moves to the DONE state.

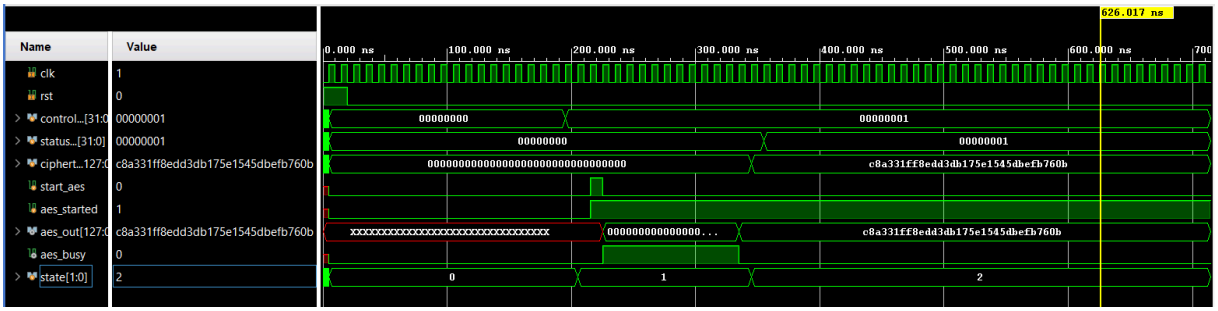


Fig. 3.3: FSM State Transition Waveform

The waveform demonstrates the transition of the Finite State Machine (FSM) from the **IDLE** state to the **RUN** state after the start command is issued through the control register. During the **RUN** state, the AES-128 encryption core performs the encryption operation while the aes_busy signal remains asserted, indicating that encryption is in progress. Once the encryption process is completed, the aes_busy signal is de-asserted, the generated ciphertext is stored in the ciphertext register, the status register is updated to indicate successful completion, and the FSM transitions to the **DONE** state.

3.4 AES Encryption Verification

The AES encryption engine was verified using standard AES-128 test vectors. The plaintext and encryption key were supplied through AXI4-Lite write transactions, and the generated ciphertext was observed after the completion of the encryption process.

Plaintext

00112233445566778899AABBCCDDEEFF

Encryption Key

000102030405060708090A0B0C0D0E0F

Generated Ciphertext

C8A331FF8EDD3DB175E1545DBEFB760B

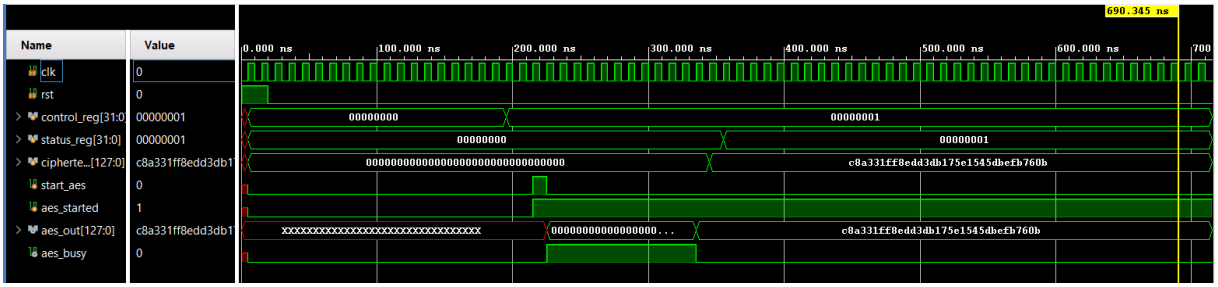


Fig. 3.4: AES Encryption Waveform

The waveform presents the complete AES-128 encryption process. A single-cycle start_aes pulse initiates the AES encryption core, after which the aes_busy signal remains asserted while the encryption operation is in progress. Upon completion, the generated 128-bit ciphertext appears at the aes_out signal and is subsequently stored in the ciphertext_reg. Finally, the status_reg is updated to indicate the successful completion of the encryption process, confirming the correct functionality of the AES encryption engine.

3.5 Simulation Results

The complete AXI4-Lite Based Cryptographic Accelerator was verified using the Xilinx Vivado simulator. The simulation confirms the successful loading of the 128-bit plaintext and encryption key through AXI4-Lite write transactions. After the start command is issued, the Finite State Machine (FSM) correctly transitions through the IDLE, RUN, and DONE states while the AES-128 encryption core performs the encryption operation. Upon completion, the generated ciphertext is stored in the ciphertext register, and the status register is updated to indicate successful execution .

```
==================================
TB Started
==================================
AES Start Command Issued
==================================
STATUS       = 00000001
FSM          = 2
AES_BUSY     = 0
AES_OUT      = c8a331ff8edd3db175e1545dbefb760b
CIPHERTEXT   = c8a331ff8edd3db175e1545dbefb760b
==================================
```

Fig. 3.5: Vivado Simulation Output

The simulation output confirms the successful operation of the proposed AXI4-Lite Based Cryptographic Accelerator. The status register is updated to **00000001**, indicating the successful completion of the encryption process. The Finite State Machine (FSM) reaches the **DONE** state (**State = 2**), while the AES_BUSY signal is de-asserted, confirming that the encryption operation has finished. The generated AES_OUT value exactly matches the value stored in the CIPHERTEXT register, verifying the correct integration and functionality of the AXI4-Lite interface, FSM controller, and AES-128 encryption core. The simulation results validate the functional correctness of the complete cryptographic accelerator.

3.6 Complete System Waveform Verification

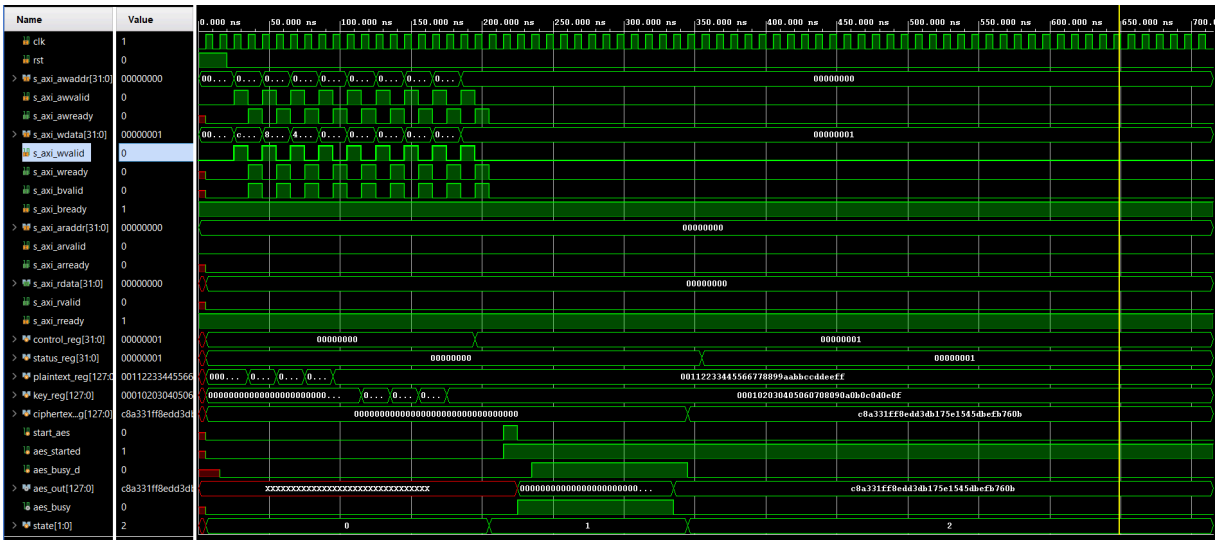


Fig. 3.6: Complete Simulation Waveform of the Proposed AXI4-Lite Based Cryptographic Accelerator

The complete simulation waveform verifies the overall operation of the proposed AXI4-Lite Based Cryptographic Accelerator from data loading to encryption completion. Initially, the plaintext and 128-bit encryption key are transferred into their respective registers through successive AXI4-Lite write transactions. After the processor writes the start command to the control register, the Finite State Machine (FSM) transitions from the **IDLE** state to the **RUN** state and generates a start pulse for the AES-128 encryption core. During the encryption process, the aes_busy signal remains asserted, indicating that the AES engine is actively processing the input data. Upon completion, the generated ciphertext appears at the aes_out signal and is stored in the ciphertext_reg. Simultaneously, the status_reg is updated to indicate successful completion, and the FSM enters the **DONE** state. The complete waveform verifies the correct interaction among the AXI4-Lite interface, internal registers, FSM controller, and AES-128 encryption core, confirming the functional correctness of the entire hardware design.

3.7 Advantages of the Proposed Design

The proposed AXI4-Lite Based Cryptographic Accelerator offers several advantages:

- Hardware-based AES encryption provides faster execution than software implementation.
- Standard AXI4-Lite interface enables seamless integration with FPGA and SoC platforms.
- Memory-mapped register architecture simplifies processor-to-hardware communication.
- Finite State Machine provides reliable control over the encryption process.

- Modular Verilog implementation supports future expansion and integration with additional cryptographic modules.
- Functional verification confirms the correctness of the design through behavioral simulation.

Conclusion:

The **Design and Verification of an AXI4-Lite Based Cryptographic Accelerator** was successfully implemented using Verilog HDL and verified through behavioral simulation in the Xilinx Vivado Design Suite. The project integrates an AES-128 encryption core with an AXI4-Lite slave interface, enabling secure hardware-based encryption using memory-mapped registers.

The developed design successfully performs plaintext and key loading, AES encryption, FSM-controlled operation, and ciphertext generation. Simulation results confirmed the correct functionality of the AXI4-Lite interface, AES core, and control logic.

The proposed architecture provides a simple, modular, and efficient hardware solution for secure embedded systems and can serve as a foundation for future FPGA implementation and advanced cryptographic applications.

References:

1. C. -H. Chen, J. -C. Ju and I. -J. Huang, "A synthesizable AXI protocol checker for SoC integration," *2010 International SoC Design Conference*, Incheon, Korea (South), 2010, pp. 103-106, doi: 10.1109/SOCDC.2010.5682961

2. Fu-ming Xiao, Dong-sheng Li, Gao-ming Du, Yu-kun Song, Duo-li Zhang and Ming-lun Gao, "Design of AXI bus based MPSoC on FPGA," *2009 3rd International Conference on Anti-counterfeiting, Security, and Identification in Communication*, Hong Kong, China, 2009, pp. 560-564, doi: 10.1109/ICASID.2009.5277006

3. R. Drehmel and H. -U. Heiss, "The Prism Bridge: Maximizing Inter-Chip AXI Throughput in the High-Speed Serial Era," in *IEEE Access*, vol. 11, pp. 50867-50883, 2023, doi: 10.1109/ACCESS.2023.3277959

4. D. Ashmawy and A. Reyhani-Masoleh, "Efficient Hardware Architectures for AES-128, AES-192, and AES-256 Encryption," in *IEEE Transactions on Computers*, doi: 10.1109/TC.2026.3701703